DOSSIER TÉCNICO

GESTIÓN DE FORMULARIOS REACTIVOS EN IONIC Y ANGULAR

Módulo: Desarrollo de Interfaces (0488) | **Curso:** 2º DAM

Resultados de Aprendizaje (RA) y Criterios de Evaluación (CE) aplicados:

- **RA 2:** Genera interfaces de usuario a partir de documentos declarativos. (CE: a, b, c - Especificación de ventanas y componentes mediante etiquetas).
- **RA 3:** Crea componentes visuales. (CE: c, d - Asignación de manejadores de eventos, validación de entrada).
- **RA 4:** Diseña interfaces usables y accesibles. (CE: a, e - Pautas de accesibilidad y adaptación a dispositivos móviles).

1. Base Teórica Técnica

1.1 ¿Qué es un formulario y su impacto en el Desarrollo Web/Móvil?

Un formulario es la estructura fundamental de interacción mediante la cual una aplicación web o móvil recolecta datos del usuario para su procesamiento. A nivel arquitectónico, un formulario actúa como la frontera entre la interfaz de usuario (UI) y la lógica de negocio (Modelo).

En el desarrollo de aplicaciones multiplataforma, los formularios tienen un impacto crítico:

- **Gestión del Estado:** Requieren sincronización constante entre lo que el usuario ve (Vista) y los datos subyacentes (Modelo).
- **Seguridad y Validación:** Son la primera línea de defensa contra la inyección de datos maliciosos o inconsistentes (Sanitización en el cliente previa a la validación en el servidor).
- **Experiencia de Usuario (UX):** En dispositivos móviles, un formulario mal diseñado provoca altas tasas de abandono. Requiere gestión precisa del teclado virtual, tamaños de área táctil (Touch Targets) adaptados a normativas WCAG, y feedback visual inmediato.

1.2 Tipos de Formularios en Angular

Angular proporciona dos paradigmas para la construcción de formularios. La elección arquitectónica debe basarse en la complejidad y escalabilidad del proyecto.

Característica	**Template-Driven Forms (Basados en Plantilla)**	**Reactive Forms (Formularios Reactivos)**
Enfoque	Declarativo (Lógica en el HTML).	Funcional/Programático (Lógica en TypeScript).
Sincronización	Asíncrona (Directiva ngModel).	Síncrona (Objetos FormControl).
Mutabilidad	Mutables. El estado se modifica directamente.	Inmutables. Cada cambio devuelve un nuevo estado.
Escalabilidad	Baja. Ideal para formularios de 2-3 campos.	Alta. Estándar de la industria para aplicaciones Enterprise.
Testing	Complejo (Requiere instanciar el DOM).	Sencillo (Se testa la lógica de TypeScript sin el DOM).

> En este módulo y en entornos profesionales, utilizaremos **Reactive Forms**. Su naturaleza predecible se integra perfectamente con flujos asíncronos (RxJS) y el nuevo sistema de reactividad basado en *Signals*.

1.3 Gestión de Formularios en Ionic (Android e iOS)

Ionic actúa como un Web Component Wrapper. Cuando desarrollamos formularios en Ionic para compilarlos nativamente (vía Capacitor/Cordova) hacia Android o iOS, debemos gestionar peculiaridades específicas del entorno móvil:

1. **Componentes UI Específicos:** No usamos el <input> estándar de HTML. Empleamos <ion-input>, <ion-select>, <ion-textarea> dentro de un <ion-item>. Esto garantiza que la renderización adopte el diseño nativo del sistema operativo (Material Design en Android, Cupertino en iOS).
2. **Gestión del Teclado Virtual (Keyboard):** Al hacer foco en un input, el sistema operativo despliega un teclado. Ionic ajusta automáticamente el viewport (Scroll Assist) para que el input no quede oculto. Es imperativo usar los atributos HTML5 type y inputmode (ej. inputmode="numeric") para desplegar el teclado adecuado (numérico, email, telefónico).
3. **Validación y Accesibilidad:** Los errores deben comunicarse visualmente (colores semánticos) y estructuralmente (atributos aria-* y etiquetas <ion-note slot="error">).

1.4 Flujo de Datos: ¿Dónde se envía la información?

Una vez que el formulario es validado en el cliente, la información extraída (form.value) debe ser persistida. En una arquitectura de aplicación móvil, el destino varía según la conectividad:

- **Almacenamiento Local (Offline First):** Para datos de sesión, preferencias o modo sin conexión. Se emplean bases de datos integradas en el dispositivo usando plugins de Capacitor como Capacitor Preferences (clave-valor simple) o Capacitor SQLite (datos relacionales complejos).
- **Backend / API REST:** Los datos se serializan a JSON y se transmiten mediante el HttpClient de Angular hacia un servidor remoto (Node.js, Java Spring Boot, etc.). El servidor valida nuevamente la carga útil y la almacena en una base de datos centralizada (PostgreSQL, MongoDB).

2. Ejemplo Práctico

El siguiente código muestra la implementación de un formulario reactivo para el registro de usuarios, integrando *Signals* para el manejo de estado de carga, y componentes nativos de Ionic.

Paso 1: Configuración del Módulo

Asegurar la importación de ReactiveFormsModule en el componente standalone o módulo correspondiente.

Paso 2: Lógica del Componente (TypeScript)

```
TypeScript
import { Component, signal } from '@angular/core';
import { CommonModule } from '@angular/common';
import { IonicModule } from '@ionic/angular';
import { ReactiveFormsModule, FormBuilder, FormGroup, Validators } from '@angular/forms';

@Component({
  selector: 'app-registro',
  templateUrl: './registro.component.html',
  styleUrls: ['./registro.component.scss'],
  standalone: true,
  imports: [IonicModule, CommonModule, ReactiveFormsModule]
})
export class RegistroComponent {
  // Definición del formulario reactivo fuertemente tipado
  public registroForm: FormGroup;

  // Uso de Signals (visto en sesiones anteriores) para gestionar el estado de envío
  public isSubmitting = signal<boolean>(false);

  constructor(private fb: FormBuilder) {
    this.registroForm = this.fb.group({
      nombre: ['', [Validators.required, Validators.minLength(3)]],
      email: ['', [Validators.required, Validators.email]],
      password: ['', [Validators.required,
Validators.pattern(/^(?=.*[A-Za-z])(?=.*\d)[A-Za-z\d]{8,}$/)]]
      // Patrón: Mínimo 8 caracteres, al menos una letra y un número.
    });
  }

  // Getter de conveniencia para acceder a los controles en la vista
  get f() { return this.registroForm.controls; }
```

```
  public async onSubmit(): Promise<void> {
    if (this.registroForm.invalid) {
      this.registroForm.markAllAsTouched(); // Fuerza la visualización de errores
      return;
    }

    this.isSubmitting.set(true);

    try {
      const formData = this.registroForm.value;
      console.log('Payload a enviar a la API REST:', formData);

      // Simulación de llamada asíncrona a un backend HTTP
      await new Promise(resolve => setTimeout(resolve, 2000));

      // Aquí se implementaría: this.httpClient.post('url', formData).subscribe(...)

      this.registroForm.reset();
    } catch (error) {
      console.error('Error al persistir los datos', error);
    } finally {
      this.isSubmitting.set(false);
    }
  }
}
```

Paso 3: Plantilla Declarativa (HTML)

HTML

```
<ion-header>
  <ion-toolbar color="primary">
    <ion-title>Registro de Usuario</ion-title>
  </ion-toolbar>
</ion-header>

<ion-content class="ion-padding">
  <!-- Enlace del FormGroup definido en TypeScript -->
  <form [formGroup]="registroForm" (ngSubmit)="onSubmit()">

    <ion-list lines="full">
      <!-- Campo Nombre -->
      <ion-item>
        <ion-input
          label="Nombre completo"
          labelPlacement="floating"
          type="text"
          formControlName="nombre"
          [errorText]="'El nombre es obligatorio (mín. 3 caracteres).'">
        </ion-input>
      </ion-item>

      <!-- Campo Email con teclado específico para móviles -->
      <ion-item>
        <ion-input
          label="Correo Electrónico"
          labelPlacement="floating"
          type="email"
          inputmode="email"
          formControlName="email"
          [errorText]="'Introduzca un email válido.'">
        </ion-input>
      </ion-item>

      <!-- Campo Contraseña -->
      <ion-item>
        <ion-input
          label="Contraseña"
          labelPlacement="floating"
          type="password"
          formControlName="password"
```

```
        [errorText]="'Mínimo 8 caracteres, letras y números.'">
       </ion-input>
     </ion-item>
    </ion-list>

    <div class="ion-padding-top">
     <!-- Botón dinámico que responde al Signal -->
     <ion-button expand="block" type="submit" [disabled]="registroForm.invalid || isSubmitting()">
       <span *ngIf="!isSubmitting()">Registrarse</span>
       <ion-spinner name="crescent" *ngIf="isSubmitting()"></ion-spinner>
     </ion-button>
    </div>

  </form>
</ion-content>
```

3. Ejercicio

Práctica: Sistema de Gestión de Inventario Móvil

Objetivo: Desarrollar la vista de inserción de productos para una aplicación de gestión de almacenes orientada a operarios móviles.

Requisitos Técnicos:

1. Construir un componente Ionic/Angular utilizando *Reactive Forms*.
2. El formulario debe capturar los siguientes datos del producto:
   - **Código de Barras (SKU):** Obligatorio, exactamente 12 caracteres alfanuméricos.
   - **Nombre del Producto:** Obligatorio.
   - **Categoría:** Menú desplegable (<ion-select>) con al menos 3 opciones (Electrónica, Herramientas, Consumibles).
   - **Stock Mínimo:** Obligatorio, tipo numérico (teclado numérico nativo), valor mayor que 0.
3. Implementar validadores personalizados (Custom Validators) si es necesario.
4. La interfaz debe mostrar mensajes de error claros usando la propiedad errorText de los componentes de Ionic.
5. Al hacer submit, simular una petición de guardado gestionando el estado con un Signal<boolean> para desactivar el formulario y mostrar un loader, seguido de un <ion-toast> que confirme el guardado local.